

Understand the code. Respect the evidence. Build to help.

A practical developer handbook for the Multi-Modal Disaster Predictor project.

Prepared for Gourav Dutta. Start with the fundamentals, trace a request through the application, reproduce two machine-learning experiments, and see exactly what must change before this becomes a public warning system.

Included	Working web app, source adapters, account settings, persistent alerts, training code and measured evaluation.
Research boundary	The live watch cards use explicit rules. Trained models predict weather proxies; they do not predict verified disasters.
Core principle	An unavailable feed is not an all-clear. A good-looking number is not evidence of predictive skill.

Emergency decisions: Follow NDMA Sachet, IMD, CWC and local authorities. Call 112 for an emergency in India. This project is educational and is not an operational warning service.

Your reading route

Read the system in the same order that information travels through it.

Pages / chapters	What you will understand
2-5	The scientific scope, architecture, folders and local setup.
6-9	React, state, routes, charts and the account interface.
10-15	Authentication, APIs, data storage, live sources, rules and notifications.
16-22	Dataset provenance, features, temporal splits, model fitting, evaluation and inference.
23-27	Multimodal expansion, deployment, debugging, verification and next milestones.
28	Sources and the final readiness checklist.

You do not need to memorise the repository. First understand who owns each responsibility. A browser component displays data; an API route checks a request; a feed adapter talks to a provider; a rule turns measurements into a research watch; the database remembers choices and alerts.

How to learn actively

- Open the named source file while reading its chapter. Find the function, then trace its input and output.
- Change a small UI label and reload. Next change a local test fixture. Only then alter a data rule.
- Keep a notebook with three columns: input, transformation and output. Most bugs fit somewhere in that chain.
- Reproduce the ML results before replacing algorithms. A reproducible modest baseline is more useful than an unexplained accuracy claim.

What “each piece” means here

This guide covers all authored product modules and their interactions. Generated dependency code is explained by responsibility rather than copied. The source archive contains the implementation, migrations, training scripts, downloaded datasets and model artifacts. The handbook explains syntax with focused extracts instead of pages of generated library internals.

Define the problem honestly

Different hazards need different evidence and different time horizons.

Hazard	Version 1 behaviour
Flood	Screens 24-hour rainfall totals and displays modelled river discharge. It does not estimate flood depth or inundation.
Drought	Counts dry days over a retrospective 30-day window. A dry week or month is not automatically drought.
Earthquake	Shows earthquakes already reported by USGS. It offers no 24-48-hour occurrence prediction.
Acid rain	Displays modelled SO2 and NO2 precursors. No rainwater pH or precipitation chemistry is inferred.
Cyclone	Shows regional GDACS storm reports and links to IMD. The proximity radius is not a storm-impact footprint.

The original ambition is valuable: warn people early enough to act. But one classifier trained on arbitrary weather columns cannot solve all five problems. A target must define what happened, where, when, at what severity and how it was verified.

Why earthquakes are different

USGS states that a reliable prediction must specify time, location and magnitude, and that major earthquakes cannot currently be predicted this way. Early warning detects an earthquake after it begins and can provide seconds of warning in some systems. That is not a two-day prediction. Prithvi therefore uses a recorded-event view, with no earthquake danger score.

Why a probability is not a confidence decoration

A number such as “82% flood risk” needs a precise outcome and calibration evidence. If 100 similar forecasts say 82%, roughly 82 verified outcomes should occur under the evaluated definition. Version 1 does not have that evidence, so it never invents this number.

Follow one complete request

Change Bhubaneswar to Kolkata and trace the consequences.

Browser city selector

- > GET /api/monitor?area=kolkata
- > validate area against AREAS
- > six independent source adapters
- > D1 source cache or fresh provider response
- > complete-window checks and makeHazards()
- > optional saved-profile alert evaluation
- > JSON response
- > cards, chart, map and timestamps

The browser owns the currently viewed city. The server owns identity, provider access, rule evaluation and durable records. These are separate responsibilities. Changing the viewing city does not silently change the user's saved alert area.

Parallel source loading

Weather, river, air, earthquake, cyclone and dry-window requests do not depend on each other. Promise.all starts them together. Each is wrapped by source(), which returns either a successful payload or an unavailable state. A failed air-quality endpoint therefore does not erase a valid weather forecast.

Reading the result

```
{
  area: { id, name, lat, lon },
  weather: { status, data, fetchedAt },
  flood: { status, data, fetchedAt },
  hazards: [{ id, level, label, metric, detail }],
  generatedAt: "an ISO UTC timestamp"
}
```

generatedAt says when the application assembled its answer. fetchedAt says when that source was retrieved. Neither automatically means the underlying numerical weather model was initialised at that time.

Know the repository

A file is easier to understand when you know which layer it belongs to.

File or directory	Purpose
app/page.tsx and route pages	Small wrappers selecting overview, history, preparedness, help or learning.
app/frame.tsx	Builds server-safe sign-in and sign-out paths; passes them to the client.
app/observatory.tsx	Main UI, fetch lifecycle, charts, account dialog, settings and inbox.
app/globals.css	Shared colours, typography, spacing and responsive layout.
lib/locations.ts	Supported city coordinates and source links.
lib/feeds.ts / risk.ts	Provider parsing and pure screening logic.
lib/storage.ts / alerts.ts	Database/cache helper and notification evaluation.
app/api/*	HTTP entry points: validate, authorise, invoke logic and return JSON.
db/schema.ts / drizzle/*	Typed table definitions and generated SQL migrations.
ml/*	Download, feature construction, training, evaluation and local inference.
components/ui/*	Prebuilt accessible UI primitives, composed by the app.
build/* / scripts/*	Framework, Worker packaging, dependency and development tooling.

The long UI module is intentionally easy to locate for this first version. As the app grows, move Forecast, History, Help, Prepare and Learn into separate files and split large route bundles. Keep network logic in lib/ rather than mixing provider URLs into button handlers.

Run it on your computer

You need Node for the website and Python for the model experiments.

```
npm ci
npm run build
node --import ./scripts/sites-env.mjs \
  ./node_modules/wrangler/bin/wrangler.js dl execute DB \
  --local --config dist/server/wrangler.json \
  --persist-to .wrangler/state \
  --file drizzle/0000_complete_maverick.sql
npm run dev
```

npm ci installs the exact versions in package-lock.json. It is more reproducible than installing whatever package version happens to be newest. The build creates the local Worker configuration. The migration creates the local tables. The final command starts development; open the URL printed in your terminal.

Windows details

The multiline example uses shell continuation characters for readability. In PowerShell, run the long database command on one line. If npm.ps1 is blocked, use npm.cmd rather than changing the machine-wide execution policy. Use Python's venv so ML packages do not conflict with unrelated projects.

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# Or call .venv\Scripts\python.exe directly
pip install -r ml/requirements.txt
python ml/download_data.py
python ml/train.py
```

The portable starter simulates identity only for loopback development. Hosted identity is supplied by the platform. Never expose a development server as a public authentication service. Copying the source to another host also requires a database binding, migrations and a suitable authentication boundary.

Read React and TypeScript

A component is a function that describes what the screen should contain.

```
function Signal({ name, value, note }: any) {
  return <div className="signal">
    <span>{name}</span>
    <strong>{value}</strong>
    <small>{note}</small>
  </div>;
}
```

JSX lets JavaScript return a tree of elements. Braces insert a JavaScript expression into that tree. `className` is React's spelling of the HTML class attribute. Props are inputs to a component; destructuring pulls named values from the props object.

Types help expose mistakes

Hazard is a TypeScript type containing `id`, `name`, `level`, `label`, `summary`, `metric` and `explanation` fields. `Level` is a union of four allowed strings: `neutral`, `low`, `caution` and `danger`. This prevents a misspelled state from silently falling through styling or alert logic. Several boundary payloads use any in this first version; a useful improvement is provider-specific runtime schemas and narrower TypeScript types.

```
const [areaId, setAreaId] = useState('bhubaneswar');
const area = AREAS.find(a => a.id === areaId);
```

`useState` returns the current value and a setter. Calling `setAreaId` asks React to render again. `find` returns the first matching area. The implementation validates IDs before accepting them, so a supported city should always be found.

Conditional rendering

A ternary chooses between two branches: `busy ? loading : result`. `Array.map` creates repeated cards. A stable key such as `hazard.id` tells React which item is which across updates. Do not use random keys, because they remount components and lose state.

State, effects and race conditions

Network requests can finish in a different order from the order they started.

```
const id = ++reqId.current;
setBusy(true);
const response = await api(`/api/monitor?area=${areaId}`);
if (id === reqId.current) setData(response);
```

Imagine selecting Kolkata, then immediately selecting Jaipur. If Kolkata's slower request finishes last, a careless implementation shows Kolkata's data under Jaipur's title. `reqId` gives every request a sequence number. Only the newest sequence may update the visible response.

Effects have a lifecycle

`useEffect` starts a refresh when the selected area changes. It also installs a 15-minute interval for the overview, then returns a cleanup function to remove that interval. Without cleanup, each navigation could leave another timer running.

What is and is not saved locally

The viewing city is a device preference. The emergency-bag checklist is explicitly labelled as saved on that device. They use `localStorage`. The authoritative alert area, opt-in state and inbox live in D1 and are connected to the signed-in user. Browser storage is not an account database.

Notifications

The app only calls `Notification.requestPermission` after a user clicks the enabling button. It sends a browser notification for a recent unread inbox alert if permission is granted. A ref tracks notifications seen during the page lifetime. This is a page-open notification mechanism; it is not a service-worker Web Push implementation.

Exercise

Throttle the network in browser developer tools. Change cities twice quickly and verify that the final title and data agree. Reload and confirm the view preference remains while account settings come from the server.

Pages, charts and accessible controls

Good interface design makes uncertainty readable.

Overview shows the primary working surface immediately: selected area, five hazard cards and a 48-hour chart. Clicking a card opens its evidence and limitations. The map is an OpenStreetMap embed with a selected-area marker; it is explicitly not a hazard footprint.

The Forecast component

Forecast receives the already parsed weather response. It sums exactly 48 hourly rainfall entries and switches the chart between rainfall and wind. Recharts draws the plot. Tooltip values keep their units, and dates are displayed in Asia/Kolkata. Missing measurements are not connected into an invented continuous series.

The History component

A year selector requests reanalysis and groups daily data into months. Rainfall totals, dry-day counts and heavy-rain-day counts answer different questions. A heavy-rain day is not a flood event. A CSV button creates a Blob, temporarily creates a download URL, clicks a download link and revokes the URL afterward.

Reusable controls

The app composes Dialog, Select, Tabs, Switch and Checkbox primitives. They provide keyboard behaviour and semantic roles. The account window uses real tab panels rather than manually hiding unrelated elements. CSS provides visible focus states, reduced-motion support and layouts for narrow screens.

Keep numbers interpretable

Every number needs a quantity, unit, window and source. “42” is meaningless. “42 mm over the next 48 hours, Open-Meteo forecast” is inspectable. Colours carry status, but text labels carry the meaning; users should not need colour vision to interpret the screen.

The remaining product pages

Preparedness, contacts and learning make the data useful.

Prepare renders five hazard-specific guidance cards and a local checklist. `lib/content.ts` holds the text and official source URLs. Keeping content separate from rendering makes it easier to review wording without altering application logic.

Help filters a small verified directory by category and search text. National numbers remain visible across all areas. Local hospital and police contacts currently cover Bhubaneswar only. Each card has a tel: link and a source link. No card claims bed availability, response time or emergency dispatch from an NGO.

Learn explains the data pipeline and model status, loads `model-report.json`, and links to this handbook and the source archive. The model report is generated by training code; the page does not type a made-up accuracy into the UI.

Useful syntax

```
CONTACTS.filter(c =>
  (c.area === 'all' || c.area === area.id) &&
  (category === 'All' || c.kind === category)
)
```

`filter` keeps items that satisfy every condition. The logical OR admits national contacts or the selected local area. The logical AND applies the category condition too. Search uses lowercased text for case-insensitive matching.

Maintenance responsibility

A sourced phone number can still change. The directory displays the date its source pages were checked. Expand it by researching official district and hospital websites, storing each provenance URL and review date, and reviewing entries periodically. Do not generate plausible-looking phone numbers.

Identity and ownership

A login button is not enough; every write must be checked on the server.

app/chatgpt-auth.ts reads identity headers supplied by the hosting platform. getChatGPTUser returns null for no identity or an object with a stable Site-scoped user ID and email. The email is contact/display information; the user ID is the ownership key.

```
const user = await getChatGPTUser();
if (!user) return Response.json(
  { error: 'Sign in required' }, { status: 401 }
);
```

The profile route does not accept user_id from the browser. It chooses the ID from trusted server identity. The inbox query filters by that same ID. This prevents a visitor from changing an ID in the request to read another person's records.

Sign-in navigation

The platform owns /signin-with-chatgpt, /signout-with-chatgpt and /callback. frame.tsx provides links built by server helpers. Sign-in uses a top-level anchor navigation, not fetch. A same-origin relative return path avoids open redirects to arbitrary websites.

Cross-origin requests

Preference saves and mark-as-read requests also compare the Origin header to the request origin. Authentication and ownership remain essential; origin validation is an additional defence. Production should also apply appropriate request limits and audit logging before broad rollout.

Private release

This Site starts private to its owner. The account flow is functional within that access policy. Public registration by other users needs an explicit audience change and a review of scheduler authentication, rate limits, privacy notices and provider quotas. Private access is not the same as an application-level membership model.

HTTP routes and validation

The API is the contract between the browser and the server.

Route	Contract
GET /api/monitor	Requires a supported area. Returns feed states and hazard summaries.
GET /api/history	Requires a supported area and integer year 2000-2025. Returns monthly reanalysis.
GET /api/profile	Returns optional user identity, saved preferences and delivery capability.
POST /api/profile	Requires identity, same origin, valid area, valid threshold and Boolean opt-ins.
GET /api/alerts	Returns only the authenticated user's latest 50 alerts.
PATCH /api/alerts	Marks only that user's unread alerts as read.
POST /api/run-monitor	Runs the service updater through configured scheduler access.

```
const area = getArea(query.get('area') || 'bhubaneswar');
if (!area) return Response.json(
  { error: 'Choose a supported area.' }, { status: 400 }
);
```

A 400 response means the request values are invalid. A 401 means authentication is needed. A 403 means a request is forbidden. A 503 means a dependency is temporarily unavailable. The client `api()` helper checks HTTP success and turns an error payload into a visible recoverable message.

Security through bounded inputs

Users select known coordinates rather than submitting arbitrary provider URLs. The server uses fixed upstream hosts. That reduces accidental unbounded requests and avoids turning a generic URL parameter into a server-side request forgery route. The year range also bounds the amount of historical data fetched.

The database and cache

Durable state belongs in tables, not in a component's memory.

Table	What it remembers
profiles	User ID, email, alert area, enabled state, threshold, email opt-in and update time.
alerts	Unique alert ID, user ownership, hazard, level, message, timestamps and email result.
feed_cache	Provider response JSON and retrieval time, keyed by source plus area or year.
monitor_runs	Background run ID, time, number of profiles checked and alerts created.

```
db.prepare('SELECT * FROM profiles WHERE user_id=?')
  .bind(user.userId).first();
```

prepare creates a parameterised statement. bind supplies values separately from SQL syntax. Avoid concatenating user text into a SQL query. Each prepare contains one SQL statement; related operations can use the D1 batch API when atomicity is required.

Schema and migrations

db/schema.ts declares the shape with Drizzle. npm run db:generate produces SQL under drizzle/. The first migration creates the four tables and an index on alerts(user_id, created_at), matching the inbox query. Generate a new migration for later changes; do not rewrite an already applied migration.

Cache lifetime

Weather and earthquakes cache for 15 minutes, river and air for an hour, cyclone reports for 30 minutes, dry-window history for 12 hours and historical years for a day. cached() checks age before reuse. Expired data is not silently labelled fresh when a provider fails; the source becomes unavailable.

Known scale limit

The first version has no cache-retention cleanup or alert archival job. Add retention, operational metrics and failure monitoring as usage grows. Consider a transactional outbox for reliable delivery rather than treating database insertion and email sending as one dependable action.

Provider adapters and data quality

A successful HTTP response can still contain unusable measurements.

lib/feeds.ts contains a json() helper with an 18-second timeout and an HTTP status check. Each adapter then checks the expected payload. Weather must provide enough future hours. River flow must contain at least one numeric discharge value. Dry-window rainfall must contain all 30 days.

The six adapters

- Weather: select the next 48 complete forecast hours from a UTC time array; carry rain, probability, temperature, humidity, wind and soil fields.
- River: keep daily modelled discharge and the grid coordinates. Never rename discharge as local flood depth.
- Air: use the current model hour for SO₂/NO₂; it is not a chemical measurement of collected rain.
- Earthquakes: query M2.5+ within 500 km and seven days; retain magnitude, time, location, depth and source URL.
- Cyclones: parse only tropical-cyclone RSS items, time-filter them and calculate distance from their reported centre.
- Dry window: use complete daily reanalysis ending five days ago, respecting recent-data latency.

Null is not zero

```
sumComplete([0, 0, 0], 3) // 0, genuinely dry
sumComplete([0, null, 0], 3) // null, unknown
sumComplete([20], 24) // null, incomplete
```

This distinction is safety-sensitive. Treating a missing rainfall hour as zero can push a dangerous total below a threshold. The tests cover these cases directly.

The rule engine

Small pure functions make assumptions easy to inspect and test.

lib/risk.ts defines finite(), sumComplete(), maxComplete(), rainfallLevel(), distanceKm() and makeHazards(). These functions do not fetch data or write records. That makes them deterministic: the same inputs produce the same outputs.

```
function rainfallLevel(mm) {
  return mm === null ? 'neutral'
    : mm >= 100 ? 'danger'
    : mm >= 50 ? 'caution' : 'low';
}
```

The rainfall signal uses the larger of the first and second 24-hour totals in the upcoming 48-hour window. It does not calculate every sliding 24-hour window. The thresholds are project screening heuristics, not IMD warning categories and not flood probabilities.

The drought card uses at least 20 days below 1 mm in the 30-day historical window as a watch. This is not seasonally normalised, so it may over-alert in a naturally dry season. A future drought model should compare against multi-decadal climatology and validated impact/soil/water indicators.

Cyclone reports within 800 km produce a regional watch, with GDACS red reports mapped to danger. This is a coarse proximity rule; it cannot replace a forecast cone or local surge warning. Earthquake and acid-rain cards remain neutral monitoring views and never emit occurrence predictions.

Next improvement

Add expert-reviewed regional trigger policies, hysteresis, event identity and evidence expiry. Evaluate them against historical events before broadening delivery. Avoid allowing arbitrary public users to choose thresholds that look like scientific policy without explaining their effect.

From a watch to a notification

An alert has to be relevant, owned, deduplicated and honestly delivered.

profile enabled?

- > hazard at caution/danger?
- > meets saved threshold?
- > insert unique alert record
- > optional configured + opted-in email
- > browser reads inbox and may notify

evaluateProfile reads the user's saved area and threshold. A danger-only profile ignores caution. A disabled profile receives nothing. The unique ID combines user, area, hazard, level and a six-hour time bucket. INSERT OR IGNORE ensures repeated page refreshes do not duplicate the same alert within that bucket.

What delivery states mean

The durable inbox is the default delivery channel. Resend is optional: both RESEND_API_KEY and a verified ALERT_FROM_EMAIL must exist, and the user must explicitly opt in. A successful provider HTTP response records sent; a failure records failed. This is provider acceptance, not proof the person read the message. The current implementation has no email retry/outbox.

Browser notifications require explicit permission and an open page. Closing the tab stops this mechanism. Closed-browser Web Push would require a service worker, VAPID configuration, subscription records and a push delivery service.

Background checks

runMonitoring groups opted-in profiles by city, fetches once per city, evaluates each profile and writes a monitor_runs row. The linked hourly task calls the private updater while the browser is closed. Verify the most recent run in account settings; a saved schedule alone does not prove a successful run. No emergency-grade timing guarantee is claimed.

Build a dataset you can explain

Data provenance is part of the model, not a footnote.

`ml/download_data.py` requests daily precipitation, mean temperature and maximum wind for Bhubaneswar, Kolkata and Jaipur, from 2000-01-01 through 2025-12-31. It writes one CSV per city and a manifest with source URL, retrieval time, row count and missing-value count.

This run downloaded 9,497 daily rows per city, for 28,491 rows in total. The training report hashes each CSV with SHA-256. If the upstream provider revises reanalysis, downloading again may produce different bytes and slightly different results. The saved files are the reproducible experiment inputs.

What reanalysis is

Reanalysis combines observations with numerical models to reconstruct past weather consistently. It is not the same as what a forecaster knew at that historical moment. Using reanalysis for a first retrospective baseline is reasonable; calling that result a verified operational 24-hour warning skill would be misleading.

Retry behaviour

The script retries a failed download up to three times with increasing short waits. It refuses to silently mix an old file with a differently requested date range. Read the manifest after a run. A network timeout should not result in a synthetic replacement dataset.

Exercise

Open one CSV and identify the date, city and units. Count missing values. Check that no date appears twice for a city. Plot one year and explain its seasonal pattern before training anything. If you cannot describe your input, you cannot interpret your model.

Features and target construction

The most important question is: was this value known at issue time?

Feature	Meaning at the end of day t
rain	Precipitation on the completed day t.
rain_3d / rain_7d / rain_30d	Rolling totals ending on day t, with all required days present.
temperature / wind	Mean temperature and maximum wind for completed day t.
month_sin / month_cos	Cyclic encoding that keeps December and January close.
latitude / longitude	City position; useful but not proof of unseen-region skill.

```
g['rain_7d'] = g.rain.rolling(7, min_periods=7).sum()
g['next_day_rain'] = g.rain.shift(-1)
future = pd.concat([g.rain.shift(-i) for i in range(1,8)], axis=1)
g['next_7d_rain'] = future.sum(axis=1, min_count=7)
```

rolling(7) uses the current and previous six rows. shift(-1) places tomorrow's observed value beside today's features, solely as a training target. It must never enter the input feature list. The seven-day target requires all seven future values; min_count prevents missing futures from becoming zero.

The second target is dry_next_week = next seven-day rainfall total below 7 mm. This is an explicitly defined weather proxy, not an official drought declaration. The feature builder sorts by city/date, rejects duplicate or non-contiguous dates, and drops rows with incomplete features or targets.

A common bug

Computing rolling totals after concatenating cities without grouping can let the end of Jaipur's history leak into the start of Kolkata's history. The script builds features inside each city group before concatenating them.

Split time before celebrating

Randomly shuffling environmental time series can make a weak model look strong.

Partition	Purpose and dates
Training	Before 2022-12-25. Fit model parameters.
Validation	2023-01-01 through 2023-12-24. Select the dry-week threshold.
Test	2024-01-01 through 2025-12-24. Evaluate once after choices are fixed.

The seven-day embargo prevents a target window from crossing into the next partition. For example, a training sample whose target includes the first week of 2023 would expose validation-period outcomes during fitting. The gap blocks that boundary overlap.

What this split does not prove

All three cities occur in training and testing. Therefore the experiment checks future dates for known cities, not generalisation to a new region. Daily target windows also overlap, so adjacent samples are correlated. Confidence intervals should be based on blocked time periods or complete events rather than pretending every row is independent.

Future evaluation

Reserve whole basins or states, not nearby pixels from the same storm. Group samples from one disaster episode together. Keep a final chronological test period untouched while tuning. Compare forecasts at the same issue times and lead times using historical forecasts available then.

Leakage checklist

- No future rainfall in feature columns.
- No event-report text written after the forecast issue time.
- No satellite image captured after the event hidden inside an embedding.
- No normalisation fitted on test rows.
- No warning threshold picked by looking at final test performance.

Train the rainfall model

A random forest combines many modest decision trees.

```
rain = RandomForestRegressor(  
    n_estimators=80, max_depth=9,  
    min_samples_leaf=20, random_state=42, n_jobs=-1  
)  
rain.fit(train[FEATURES], train.next_day_rain)  
pred = rain.predict(test[FEATURES])
```

Each tree partitions the feature space with questions such as whether the previous seven-day rainfall is above a learned value. A leaf predicts an average training target. The forest averages trees, reducing the variability of a single tree.

`n_estimators` controls the number of trees. `max_depth` limits complexity. `min_samples_leaf` prevents leaves based on a handful of examples. `random_state` makes stochastic choices reproducible. `n_jobs=-1` allows parallel CPU work. These settings are a modest baseline, not a claim of optimal tuning.

Measured result

Metric	Held-out 2024-2025 result
Mean absolute error	3.425 mm/day
Root mean squared error	6.868 mm/day
Persistence baseline MAE	3.739 mm/day
Test rows	2172

MAE is the average absolute difference between forecast and observed next-day rain. RMSE penalises large misses more strongly. Beating persistence on overall MAE does not establish skill at rare extreme rainfall, flooding or evacuation decisions. Inspect the largest errors and event-specific recall separately.

Train the dry-week classifier

A classifier answers the label you supplied, not the disaster you wish it understood.

The second forest estimates the probability that the next seven-day rainfall total is below 7 mm. It learns from the same causal feature columns. `predict_proba` returns one probability per class; `[:, 1]` selects the positive class probability.

```
cost = 3 * false_negatives + false_positives
threshold = candidate_with_lowest_validation_cost
```

The script checks thresholds from 0.10 through 0.90 on validation data. Missing a dry-week example receives three times the cost of a false watch. This is a documented learning choice, not a public safety policy. A real alert policy needs expert input, public impact analysis and independent validation.

Metric	Held-out result
Chosen threshold	0.25
Average precision / PR-AUC	0.908
Brier score	0.112
Precision	0.723
Recall	0.966

These are dry-week metrics, not drought prediction accuracy. The low threshold deliberately catches many positive examples and also produces false positives. Seasonal patterns and overlapping windows can make this task easier than predicting the onset of a damaging drought.

Calibration

A model's probability output is not automatically calibrated. Evaluate reliability curves, calibrate on a separate suitable validation set and compare Brier score with a climatology baseline. Do not label these probabilities as calibrated disaster risk.

Read the confusion matrix

A high recall can coexist with a substantial false-alarm burden.

Actual / predicted	Not dry	Dry
Not dry	717	393
Dry	36	1026

The model found 1026 dry-week examples and missed 36. It also flagged 393 examples that were not dry under the defined target. Precision asks how many positive predictions were correct. Recall asks how many actual positives were found.

```
precision = TP / (TP + FP)
recall    = TP / (TP + FN)
Brier     = mean((probability - outcome) ** 2)
```

Average precision summarises precision-recall behaviour across thresholds and is informative when class balance matters. Accuracy alone can hide failure on rare hazards: predicting “no event” every time may score well when events are uncommon.

Evaluate the warning, not just the row

A useful warning system needs event-level detection, time to first warning, missed events, false alarms per area per month and enough lead time to act. Repeated positive daily rows from the same episode should not be counted as dozens of independent successful disaster predictions.

Files to inspect

model-report.json contains the summary, dates, data hashes, package versions and limitations. heldout_predictions.csv contains predictions beside outcomes, city and date. Sort by absolute rainfall error; inspect monsoon extremes and compare errors by city. The report also stores per-city MAE.

Inference and reproducibility

Training creates an artifact; inference applies that artifact to the same feature contract.

```
python ml/predict.py \  
  --model ml/artifacts/rain_model.joblib \  
  --csv ml/artifacts/feature_example.csv
```

The script loads the saved model bundle, selects the exact saved feature names in their saved order, rejects missing values and calls `predict` or `predict_proba`. A reordered NumPy array can silently change meaning; selecting named DataFrame columns reduces that risk.

Do not serve this baseline blindly

The research features use completed-day reanalysis. The live dashboard uses a forecast API and recent reanalysis with latency. Wiring the saved model into the live alert path without resolving this issue-time and feature-distribution mismatch would be invalid. First build a production feature service and evaluate archived real-time predictions.

Security

joblib uses pickle-based serialization, which can execute code during loading. Load only artifacts you produced or trust. A web endpoint that accepts arbitrary uploaded .joblib files would be unsafe. The current app serves the report as JSON and does not load Python models inside the Worker.

Reproduce a run

- Use the saved input CSVs and compare their SHA-256 hashes to the report.
- Use the recorded Python and scikit-learn versions if exact replication matters.
- Run the feature and split functions unchanged.
- Keep the random seed and estimator settings.
- Compare predictions and metrics; investigate any differences before interpreting improvements.

Grow into a multimodal model

Connect modalities only when they describe the same place, time and forecast issue.

Modality	Potential evidence and key problem
Time series	Rain, river flow, soil moisture, wind and pressure. Ensure issue-time availability and consistent units.
Satellite imagery	SAR flood extents, cloud fields, vegetation and soil indicators. Align capture time and geography; handle cloud/missingness.
Static geography	Elevation, slope, drainage, land use and coast distance. Choose the correct spatial resolution.
Official text	Structured bulletins and warnings. Retain issue time, validity interval and covered regions.
Precipitation chemistry	Measured rainwater pH and ionic composition for acid deposition. Air pollutants alone cannot provide the target.

The delivered app is multi-source. It does not yet train image or text encoders. A future model could concatenate a time-series embedding, satellite embedding and structured bulletin features, with explicit masks when a modality is missing. More modalities help only if they add reliable, timely information.

The extension script

`ml/train_hazard.py` accepts verified binary labels and numeric `feature_` columns, including precomputed embeddings. It holds out a named region and dates after a cutoff, rejects inadequate class support and writes a research report. It deliberately disallows earthquake occurrence as a target. It is an untrained extension, not a set of finished disaster models.

Ground truth first

Flood labels should come from verified inundation/gauge/impact records under a clear definition. Drought labels need a chosen meteorological, agricultural or hydrological definition. Cyclone labels need track, wind/impact thresholds and forecast windows. Acid-rain labels need precipitation chemistry. Store evidence and label provenance with every example.

Keys, hosting and background access

Keep credentials on the server and access decisions explicit.

Setting	Purpose
RESEND_API_KEY	Optional email provider secret. Required only for email delivery.
ALERT_FROM_EMAIL	Verified sender address or sender identity for that provider.
MONITOR_SECRET	Optional bearer secret for a separate scheduler.
PRIVATE_SITE_MONITOR	Only true behind confirmed owner-private platform access. Turn off before making the site public.

The included public feeds do not require keys for eligible non-commercial use. That does not mean unlimited use or an emergency-service SLA. Review quotas and licensing before commercial rollout. The app does not need an OpenAI key because its current monitoring and ML pipeline do not use an LLM.

Hosted source and database

The project builds to a Cloudflare-compatible Worker plus static assets. Logical D1 bindings are declared in `.openai/hosting.json`. The hosting platform supplies the real resources, applies migrations and injects runtime settings. Do not commit real secrets into that manifest or into client-facing `NEXT_PUBLIC` variables.

Scheduled caller

The private background updater is reached using the platform's Site service access, not a user's browser cookie. It reads only public providers and stored opt-in profiles. If the Site becomes public, the private boundary is gone: disable `PRIVATE_SITE_MONITOR` first and require a properly configured service credential. An hourly task is not a guarantee of timely disaster delivery.

Debug by following the evidence

Separate a rendering issue from a provider issue and a model issue.

Symptom	First checks
Every feed unavailable	Check database binding/migrations, then provider networking and server logs. Cache access is part of every adapter.
Only one source unavailable	Inspect its HTTP status, schema, units and coverage; other feeds should remain usable.
Old city data after switching	Check request sequence protection and effect dependencies.
Settings will not save	Inspect 401/403/400/503, server identity, Origin and validation values.
No alerts	Check enabled state, saved area, threshold, source freshness, six-hour deduplication and latest monitor run.
No email	Check user opt-in, verified sender, provider key and email_state. Inbox delivery is separate.
Training fails	Check downloaded CSVs, contiguous dates, missing values, feature columns and split sizes.
Suspiciously excellent ML score	Inspect future leakage, overlapping event groups, geographic leakage and target construction.

Browser tools

Use the Network panel to inspect the request and JSON response. Use Console for component errors. Never paste secret-bearing headers or environment dumps into public bug reports. Copy the smallest reproducible request with sensitive values removed.

Server logs

Adapters log their source key and error, so you can distinguish weather:kolkata from air:kolkata. Add correlation IDs and structured operational metrics as a next step. Keep user email and identity out of unnecessary logs.

Tests that protect the meaning

Test the dangerous failure modes, not just whether a page renders.

```
node node_modules/typescript/bin/tsc --noEmit
node tests/risk.test.mjs
npm run build
```

Type checking catches mismatched shapes and invalid assumptions in code. The risk test checks missing values, incomplete windows, exact threshold boundaries and the absence of false all-clear states. The production build checks that framework and Worker output can be generated.

Tests included

sumComplete returns null for a missing hour and zero for a complete zero series. rainfallLevel crosses at 50 and 100 mm exactly. All-unavailable input yields neutral hazard states. Earthquakes remain a monitoring view. Distance between identical coordinates is zero.

Next integration tests

- Authenticate two different users and prove neither can read or mutate the other's settings or inbox.
- Run the monitor twice for one triggered profile and verify only one alert in the same six-hour bucket.
- Simulate provider timeouts and malformed data. Verify visible unavailable states without invented fallback values.
- Simulate email acceptance, rejection and timeout; add a transactional outbox before reliable retries.
- Check desktop and mobile layouts, keyboard focus, screen-reader labels and 200% text zoom in a real browser.

A successful build is not proof of visual correctness, scientific validity or delivery reliability. Keep these checks separate in your project report. The current environment allowed code/build and HTTP checks; a complete interactive browser review still belongs in your release workflow.

A practical development roadmap

Build the next version around a measurable question.

Milestone 1: a flood study for one basin

Choose a specific basin and outcome such as gauge threshold exceedance in the next 24 hours. Gather issued forecasts, upstream rainfall, river observations, terrain and verified exceedance labels. Compare a rule, persistence and an ML baseline on held-out monsoon seasons. Report missed events and false alarms, not just overall accuracy.

Milestone 2: meaningful drought monitoring

Compute SPI/SPEI or another justified index over a long climatology, choose a regional seasonal baseline, add soil-moisture and vegetation evidence, and validate against an explicitly defined drought outcome. Evaluate weekly or monthly horizons appropriate to the process.

Milestone 3: reliable delivery

Add a durable outbox, retries with idempotency, unsubscribe controls, evidence expiry, per-event deduplication, audit trails and delivery-health monitoring. Integrate approved official warning feeds. Exercise the complete flow with consented test users before a public pilot.

Milestone 4: independently reviewed pilot

Work with domain experts and a local authority or research group. Define what users should do when a watch appears, how uncertainty is communicated, and how mistakes are investigated. A research demo should not silently become emergency infrastructure.

How to present this in an interview

Explain the real engineering achievements: independent source failure handling, explicit uncertainty, persistent user ownership, time-safe features, chronological evaluation and measured baselines. Explain the limits just as clearly. Being precise about what your system cannot predict demonstrates good ML judgement.

Sources and readiness checklist

Keep the source close enough that another person can inspect it.

Weather forecast / variables

<https://open-meteo.com/en/docs>

Historical reanalysis

<https://open-meteo.com/en/docs/historical-weather-api>

Modelled river discharge

<https://open-meteo.com/en/docs/flood-api>

Air-quality forecasts

<https://open-meteo.com/en/docs/air-quality-api>

Earthquake API

<https://earthquake.usgs.gov/fdsnws/event/1/>

Earthquake prediction limits

<https://www.usgs.gov/faqs/can-you-predict-earthquakes>

Cyclone reports / IMD

<https://rsmcnewdelhi.imd.gov.in/>

Global disaster feed

<https://www.gdacs.org/xml/rss.xml>

Official Indian warnings

<https://sachet.ndma.gov.in/>

Acid deposition science

<https://www.epa.gov/acidrain/what-acid-rain>

Emergency numbers

<https://www.india.gov.in/directory/helpline>

AIIMS Bhubaneswar contacts

<https://aiimsbhubaneswar.nic.in/contact-us/>

Goonj relief contact

<https://goonj.org/donate/campaign/rahat>

Before calling it an operational system

Verified targets; issue-time features; independent regions and years; calibration; event-level evaluation; expert-reviewed warning thresholds; official advisory integration; tested background delivery; privacy and retention policy; accessible UI; contact maintenance; failure monitoring; a clear incident-response owner.

Version 1 does not satisfy that operational checklist. It is a working foundation for learning and careful development. The most useful next step is to validate one clearly defined hazard in one region, then expand on evidence.